

OpenBedside Handbook

An open-source gateway between bedside medical devices and the hospital EHR.

IV infusion pumps first. Built to take ventilators and other devices.

Version 0.2.0 · Daniel C. Pettus · Apache License 2.0

Every guide in the kit, in one file. The Markdown files in the kit are the source.

Not a medical device. Not for clinical use. Use on test networks only. See the Disclaimer at the end.

Contents

1. [Read me](#)
2. [Start here: install and run](#)
3. [User guide](#)
4. [Vendor integration guide](#)
5. [Mapping worksheet](#)
6. [Adapter reference](#)
7. [HL7 interface](#)
8. [Architecture](#)
9. [Decisions](#)
10. [Contributing](#)
11. [Disclaimer](#)

OpenBedside

An open-source gateway between bedside medical devices and the hospital's EHR. Infusion pumps first. Built so it can be extended to ventilators and other devices.

Not a medical device. Not for clinical use. See [DISCLAIMER.md](#).

Why I built it

My reason is a selfish one, and I would rather say so up front.

I am building **AI MedAgent**, a medication reasoning engine (aimedagent.net; the public demonstration runs on a simulated patient). A reasoning engine is only as good as what it can see, and it needs two feeds: the EHR and the bedside devices.

The EHR half has a door now. Federal certification rules require certified EHR technology to offer a standardized FHIR API (ONC, 45 CFR 170.315(g)(10)). Access is not free and not always quick, but the door exists and it looks much the same from one EHR to the next.

The device half has no door. Every infusion pump that reports to an EHR does it through its manufacturer's own gateway, in its own protocol, and each one is closed. IV infusions are one of the most important feeds a medication reasoning engine can have, because that is where the drug actually enters the patient, and they are the hardest data to get.

So I am doing the selfish thing in the most useful way I know: opening the device half for everyone, me included. OpenBedside is a **free, open-source device gateway. No fees, no royalties, no permission needed.** It starts with IV infusion pumps because that is the feed I need most, and it is built to take ventilators and other devices next.

OpenBedside contains no AI MedAgent code and no clinical reasoning. It is plumbing. It sends standard HL7 to any hospital system that wants it. AI MedAgent is one possible consumer of that data, and not a required one.

What it is

Every infusion pump that connects to an EHR today does it through its manufacturer's own gateway: years of work, sold with the pump, closed, and each one different. OpenBedside is one gateway, open to everyone. The parts are the same for every device:

- **the internal logic:** a common data model, state tracking, and events worked out from what the device reports;
- **store-and-forward:** every message saved to disk before it is sent, delivered in order, retried until the hospital acknowledges it;

- **EHR connectivity:** standard IHE Devices HL7 v2 messages over MLLP, the way hospital interface engines already expect them;
- **device registration and patient association:** nothing is sent until a device is registered, and each device's patient comes only from sources the hospital allows (the device itself, the ADT feed for its bed, or a person), with conflicts withheld;
- **a web page:** a live view of every device, connection, event and message, plus the Devices and Patients pages for registration and association.

The only part anyone writes for a new device, whether a manufacturer, a hospital integration team or a researcher, is a **device adapter**: a short piece of Python that translates that device's protocol into OpenBedside's data model. Every pump speaks differently, and they all carry the same facts. The adapter is the translation.

Open source, for anyone

Nothing here is proprietary. OpenBedside is released under the [Apache License 2.0](#). Anyone may download it, use it, test it, modify it, build it into their own product, and extend it to other devices. Improvements are welcome. See [CONTRIBUTING.md](#).

It was designed from public sources: the IHE Devices Technical Framework, ISO/IEEE 11073 nomenclature as published, FDA guidance, and experience. No vendor's protocol or code is in it. The worked example uses a fictional pump with an invented protocol.

Start here

You want to	Read
install it and see it run in 15 minutes	START-HERE-install-on-your-Mac.md (Linux is the same; Windows notes in the user guide)
run it, read the status page, change settings	docs/USER-GUIDE.md
connect your own device	docs/VENDOR-INTEGRATION-GUIDE.md
know exactly what the EHR receives	docs/HL7-INTERFACE.md
understand the design	docs/ARCHITECTURE.md and docs/DECISIONS.md
read it on paper or pass it on	docs/pdf/ : every guide as a PDF, and the OpenBedside Handbook with all of them in one file

The short version

```
python3 -m venv ~/.venvs/openbedside
source ~/.venvs/openbedside/bin/activate
pip install -e ".[dev]"          # from inside this folder
openbedside demo                # gateway + pump simulator + stand-in EHR, one window
# then open http://127.0.0.1:8080
```

Needs Python 3.11 or newer and nothing else. No database server, no web framework, no third-party packages at run time.

What it does in 0.2

Piece	Status
Data model: device, modules, channels, primary and secondary sources	working
Pump simulator: two modules, piggyback, rate change, KVO	working
Events worked out from observations: start, stop, complete, rate change, KVO, secondary to primary	working, each labelled <i>derived</i>
Store-and-forward (SQLite): ordered, retried, survives restarts	working
EHR outbound: IHE PCD-01 0RU^R01 over MLLP, application ACK required	working
Web page: Activity, Devices (registration), Patients (association), sign-in	working
Device registry: vendor + device id, EHR identifier (serial, asset tag, EUI-64), location, allowed patient sources	working
ADT inbound (A01, A02, A03, A04, A08, A11, A12, A13, A40): census of identifiers and locations	working
Patient association: device-reported, manual, ADT by bed; conflicts withheld; mid-infusion changes need confirmation	working
PID always sent (Unknown when unassociated); PV1 with location and visit	working
Adapter kit: base class, worked example, checker, starter-folder generator	working
Orders inbound: IHE PCD-03 RGV^015	simulator only , see the disclaimer
PCD-10 infusion event messages	planned

Piece	Status
Ventilators	registration accepts them; data model planned (the vendor guide explains what changes)
Encryption (MLLP, web page, database) and access audit log	not yet: synthetic data only

Commands

<code>openbedside demo</code>	everything in one window
<code>openbedside run -c config.toml</code>	the gateway
<code>openbedside ehr-sim</code>	a stand-in hospital interface engine
<code>openbedside check -c config.toml</code>	check an adapter's output before a hospital sees it
<code>openbedside new-adapter NAME</code>	write a starter adapter folder for a device
<code>openbedside fake-pump</code>	the fictional ACME pump used by the worked example
<code>openbedside send-order ...</code>	send a test order to the simulator
<code>openbedside send-adt ...</code>	send a test ADT message (synthetic identifiers only)
<code>openbedside set-password</code>	set the admin password for the Devices and Patients pages

Daniel C. Pettus. Apache License 2.0.

Start here: install and run OpenBedside

About 15 minutes the first time. You type the commands in **Terminal** (Applications > Utilities > Terminal). Copy each line exactly. Linux is the same; Windows notes are in [docs/USER-GUIDE.md](#).

Step 1. Do you already have a new enough Python?

```
python3 --version
```

- **Python 3.11** or higher (3.12, 3.13, 3.14): skip to Step 3.
- 3.9 or 3.10, "command not found", or a box offering to install developer tools: do Step 2. (Cancel the pop-up box if you see one.) Most Macs ship with 3.9, which is too old.

Step 2. Install Python from python.org

1. Go to **python.org/downloads/macos** in your browser.
2. Under the latest Python 3 release (3.14.7 as of August 2026), download the **macOS 64-bit universal2 installer**. It runs on Intel and Apple Silicon Macs.
3. Open the downloaded `.pkg` and click through the installer.
4. When it finishes, a Finder window opens. Double-click **Install Certificates.command**.
5. **Quit Terminal completely (Cmd-Q) and open it again**, then check:

```
python3 --version
```

Step 3. Set up OpenBedside (once)

The Python environment lives in your home folder, not inside the OpenBedside folder. If your Documents folder syncs through iCloud, this keeps thousands of small files and a live database out of the sync.

```
python3 -m venv ~/.venvs/openbedside
source ~/.venvs/openbedside/bin/activate
pip install -e "$HOME/Documents/OpenBedside[dev]"
cd ~/Documents/OpenBedside && pytest && cd ~
```

- After `source`, your prompt starts with `(openbedside)`. That means you are in it.
- `pytest` should end with **36 passed**.
- If you put the OpenBedside folder somewhere other than Documents, change the path in the last two lines.

Step 4. See it run: one window

```
openbedside demo
```

Open **http://127.0.0.1:8080** in your browser. You will see a simulated two-module pump sending to a stand-in hospital system, and the window prints one line for every message the hospital accepts. After three seconds the demo sends a made-up ADT admit of patient TEST0001 into the simulator's bed, ICU 101 A, and the pump picks up its patient. Look at the **Patients** page. Within a couple of minutes: a rate change on module A, and the piggyback on module B finishing with the primary taking over by itself.

Ctrl-C stops it.

Step 5. The same, in separate tabs (to see every HL7 message)

Open tabs with **Cmd-T**. In **each** tab, first:

```
cd ~
source ~/.venvs/openbedside/bin/activate
```

Tab	Command	What it is
1	<code>openbedside ehr-sim</code>	the stand-in hospital; prints every message in full
2	<code>openbedside run</code>	the gateway and pump simulator; page on :8080
3	<code>openbedside send-order --module SIM-0001-A --drug heparin --rate 10 --vtbi 100</code>	a test order; answers "AE module is infusing", which is correct

Step 6. See a device integration: the fictional ACME pump

Tab	Command
1	<code>openbedside fake-pump</code>
2	<code>openbedside ehr-sim</code>
3	<code>openbedside run -c ~/Documents/OpenBedside/config/acme-example.toml</code>

The page shows the **ACME** adapter connected, but nothing is sent: the pump is not registered. Go to **Devices**, click **Register** next to ACME-7731, fill in a unit, room and bed if you like, and **Save**. Messages start flowing. Stop the fake pump with Ctrl-C and watch the connection drop and, after 30 seconds, a *communication-lost* event. Start it again and the adapter reconnects by itself.

To check an adapter without any hospital system:

```
openbedside check -c ~/Documents/OpenBedside/config/acme-example.toml --seconds 20 --show-hl7
```

To give the ACME pump a patient by ADT, tick "ADT: the patient in this device's bed" when you register it, then in a spare tab:

```
openbedside send-adt --event A01 --mrn TEST0042 --unit ICU --room 204 --bed 1
```

using the unit, room and bed you registered.

Step 7. Connect your own device

Read [docs/VENDOR-INTEGRATION-GUIDE.md](#). It starts with `openbedside new-adapter yourname`, which writes a starter folder for you.

Every time after today

```
cd ~
source ~/.venvs/openbedside/bin/activate
openbedside demo
```

If macOS asks whether Python may accept incoming network connections, **Deny** is fine. Everything runs on your own computer.

User guide

How to run OpenBedside, read the interface activity page, change settings, and fix the usual problems. For first-time installation, start with [START-HERE-install-on-your-Mac.md](#).

Three ways to run it

Command	What runs	Use it for
<code>openbedside demo</code>	gateway, pump simulator and a stand-in EHR, all in one window	seeing it work; showing someone
<code>openbedside ehr-sim plus</code> <code>openbedside run</code>	the same, in two windows, so you can watch every HL7 message arrive	testing and development
<code>openbedside run -c your.toml</code>	the gateway with your own adapters and a real (test) interface engine	integration work

Before any of them, in each Terminal window or tab:

```
cd ~  
source ~/.venvs/openbedside/bin/activate
```

Stop anything with **Ctrl-C**.

The web page

Open **<http://127.0.0.1:8080>** while the gateway runs. It refreshes every two seconds and has three pages, on the menu at the top: **Activity**, **Devices** and **Patients**.

Who can change things

Anyone who can open the page can look. Registering devices and associating patients needs one of:

- **no admin password set** (the default): only a browser on the same computer as the gateway can make changes. The page says so in a banner;
- **an admin password**: anyone who signs in (button at top right). Set it once:

```
openbedside set-password          # or: openbedside set-password -c your.toml
```

Viewers who are not signed in see patient identifiers masked to their last four characters. Keep the page on localhost regardless: there is no encryption yet.

Activity

OpenBedside

Activity

Devices

Patients

this computer: changes allowed v0.2.0

Not a medical device. Not for clinical use. Test data only: use synthetic patient identifiers, this release has no encryption. Auto-programming works against the simulator only.

No admin password is set, so changes are allowed only from a browser on this computer. Set one with `openbedside set-password`.

STATUS

EHR link: **connected** Queue: sent 2

DEVICE CONNECTIONS (ADAPTERS)

Adapter	Connection	Messages	Skipped	Last error
acme-json (example)	connected	9	0	

DEVICES

Device	Patient	Module	Status	Actual rate	Sources
BIO-004512 ACME (fictional) ACME-2 · ICU 204 1	TEST0042 adt-location	ACME-7731-L	infusing	12 mL/h	primary heparin 12 mL/h (1200 [iU]/h) VTBI 250 mL, remaining 246.8 mL, delivered 3.2 mL
BIO-004512 ACME (fictional) ACME-2 · ICU 204 1	TEST0042 adt-location	ACME-7731-R	infusing	100 mL/h	primary sodium chloride 0.9% 75 mL/h VTBI 500 mL, remaining 500 mL, delivered 0 mL secondary cefazolin 100 mL/h VTBI 50 mL, remaining 23.333 mL, delivered 26.667 mL

EVENTS

When (UTC)	Event	Where	Detail
17:46:26	patient-associated <i>derived</i>	ACME-7731	{ "patient": "TEST0042", "source": "adt-location" }
17:46:23	delivery-start <i>derived</i>	ACME-7731-R	{ "note": "first observation already infusing" }
17:46:23	delivery-start <i>derived</i>	ACME-7731-L	{ "note": "first observation already infusing" }

OUTBOUND TO EHR

#	Control id	Status	Tries	Last result
2	PD2026092617462600002	sent	1	AA
1	PD2026092617462300001	sent	1	AA

ORDERS RECEIVED (PCD-03)

None.

Status. Whether the link to the EHR is up, the last connection error if not, and how many messages are pending, sent or rejected.

Device connections. One row per adapter: connected or not, messages received, messages skipped (could not be translated), and the last error. The first place to look when a device does not appear.

Devices. One row per pump module of every registered device: the identifier sent to the EHR, its location, its patient and where that came from, status, the rate it is actually running, and each source hanging on it (primary and, when running, secondary) with drug, programmed rate, dose rate, VTBI, volume remaining and delivered.

Events. What happened, newest first: delivery start, stop and complete, rate changes, KVO, secondary to primary, communication lost and restored, and the patient decisions (associated, cleared, conflict, review). Every one is labelled *derived*, because OpenBedside works them out

rather than the device reporting them, and says so.

Outbound to EHR. Every message queued for the hospital: its control id, whether it was sent, how many tries it took, and the hospital's reply (AA means accepted).

Orders received. Test orders sent to the simulator and how they were answered.

The same information is available as JSON at <http://127.0.0.1:8080/api/state> for anyone who wants to build their own view.

Devices: registration

Nothing from a device reaches the EHR until it is registered.

Seen but not sent lists every device an adapter can hear that is not registered, is retired, or is registered as a different type (a ventilator registration will not carry pump data). Click **Register** and the form fills in what the device reported.

OpenBedside

Activity

Devices

Patients

this computer: changes allowedv0.2.0

Not a medical device. Not for clinical use. Test data only: use synthetic patient identifiers, this release has no encryption. Auto-programming works against the simulator only.

No admin password is set, so changes are allowed only from a browser on this computer. Set one with `openbedside set-password`.

SEEN BUT NOT SENT

Devices an adapter can hear that are not registered, are retired, or are registered as a different type. Nothing from them reaches the EHR.

None.

REGISTERED DEVICES

Type	Vendor	Device id	EHR id	Location	Patient from	Status	
Infusion pump	ACME (fictional)	ACME-7731 ACME-2	BIO-004512 asset	ICU 204 1	device, ADT, manual	active	EditRetire

REGISTER A DEVICE

Device type

Infusion pump

Vendor (as the adapter reports it)

ACME (fictional)

Device id (serial, as the device reports it)

ACME-7731

Model

ACME-2

Hospital asset tag

BIO-004512

EUI-64 (16 hex characters, if it has one)

Send to the EHR as (what the nurse scans)

Asset tag

Unit (point of care)

ICU

Room

204

Bed

1

Facility

Connection binding (only for devices that never send an id)

adapter.port

Find this device's patient from

☒ the device (wristband scanned at the device)

☒ ADT: the patient in this device's bed

☒ a person, on the Patients page

Notes

SaveClear

Saved.

Field	What it is for
Device type	infusion pump, ventilator, other. Must match what the adapter delivers
Vendor, device id	how the device identifies itself. Together they are the key: two vendors can both ship serial 12345
Asset tag, EUI-64	the hospital's tag, and the device's 64-bit id if it has one

Field	What it is for
Send to the EHR as	what the nurse scans to tie the device to a patient: serial, asset tag or EUI-64. If this does not match the label, device data never lands in a chart
Unit, room, bed, facility	where the device lives. Sent in PV1-3, and needed for association by ADT
Find this device's patient from	which sources may name the patient: the device, ADT for this bed, a person
Connection binding	only for devices that never send their own id, such as a serial device behind a terminal server port

Retire stops a device being sent without deleting its history. Identity is never an IP address.

Patients: association

Each registered device's patient is decided from the sources its registration allows:

- **device**: the device reported an identifier (a wristband scanned at the pump);
- **manual**: someone associated them on this page;
- **ADT for this bed**: the hospital's ADT feed has exactly one patient in the device's registered bed.

State	Meaning	What the EHR receives
associated	the allowed sources agree	the patient, and PV1 with their location and visit
none	no source has a patient	PID-3 Unknown
conflict	sources disagree, or two patients in one bed	PID-3 Unknown until fixed
review	ADT alone would change the patient while the device is running	PID-3 Unknown until someone clicks Confirm new patient

OpenBedside
Activity
Devices
Patients
this computer: changes allowed
v0.2.0

Not a medical device. Not for clinical use. Test data only: use synthetic patient identifiers, this release has no encryption. Auto-programming works against the simulator only.

No admin password is set, so changes are allowed only from a browser on this computer. Set one with `openbedside set -password`.

DEVICE TO PATIENT

Every source allowed for a device must agree, or no patient is sent. A patient change from ADT alone while a device is running waits for a person to confirm.

Device	Location	Association	Patient	Detail
BIO-004512 ACME (fictional)	ICU 204 1	associated	TEST0042 adt-location	from adt-location

ASSOCIATE A DEVICE WITH A PATIENT

Device
Patient identifier (MRN)
Assigning authority

BIO-004512 (ACME (fictional))
default from config
Associate

CENSUS FROM ADT

Patient	Visit	Class	Location	Last event
TEST0042 HOSP	V42	I	ICU 204 1	A01

Recent ADT: A01 ***0042 at ICU 204 1

A manual association stays until it is cleared. Clear it when the device comes off the patient, or it will conflict with the next one.

Census from ADT is what the hospital's ADT feed has told the gateway: identifiers, visit, class and location. Names are never stored.

To try it without a hospital:

```
openbedside send-adt --event A01 --mrn TEST0002 --unit ICU --room 101 --bed A
openbedside send-adt --event A02 --mrn TEST0002 --unit ICU --room 102 --bed B # transfer
openbedside send-adt --event A03 --mrn TEST0002 # discharge
```

Use made-up identifiers only. This release is not for real patient data.

Settings

Every setting, with its default, is in [config/example.toml](#). Copy it somewhere outside iCloud-synced folders if you like, edit the copy, and run `openbedside run -c path/to/copy.toml`. The ones most often changed:

Setting	Default	Change it when
<code>[ehr] host , port</code>	127.0.0.1, 6661	pointing at a real (test) interface engine
<code>[ehr] receiving_app , receiving_facility</code>	EHR, HOSP	the interface engine expects specific MSH-5 and MSH-6 values
<code>[gateway] periodic_seconds</code>	60	the hospital wants snapshots more or less often

Setting	Default	Change it when
[gateway] offline_after_seconds	30	your device reports less often than every 30 seconds
[gateway] database	openbedside.db	you want the queue somewhere specific
[ui] port	8080	something else already uses 8080
[orders] enabled	true	set false with real devices; orders are simulator-only
[registry] unknown_devices	hold	send sends unregistered devices with patient Unknown (lab use only)
[patients] assigning_authority	HOSP	the hospital's code for PID-3.4
[adt] host , port	127.0.0.1, 6663	the interface engine sends ADT somewhere else
[adt] enabled	true	no ADT feed

Files it creates

openbedside.db (plus `-wal` and `-shm` files while running) in the folder you start it from. It holds the outbound queue, the event log, the device registry, the ADT census, manual associations and the admin password hash. Delete it to start clean: pending messages, registrations and the password go with it.

Problems and fixes

You see	Cause	Fix
command not found: openbedside	the environment is not active in this window	<code>source ~/.venvs/openbedside/bin/activate</code>
no such file or directory: .../activate	the environment was never created	<code>python3 -m venv ~/.venvs/openbedside</code> , then install again
Browser: unable to connect	the gateway is not running, or stopped with an error	look at the gateway's window
A port is already in use	another copy is running, or another program uses the port	stop the other copy, or change the port in your config
EHR link: not connected	nothing is listening at [ehr] host:port	start <code>openbedside ehr-sim</code> , or check the interface engine address

You see	Cause	Fix
Messages stuck <i>pending</i>	the EHR is down or not acknowledging	fix the link; they send in order once it is back
A message <i>rejected</i>	the receiver replied AR	read "last result"; the queue moves on, the message is kept
Adapter shows skipped messages	your device sent something the adapter could not translate	run <code>openbedside check</code> and read the warnings
AE module is infusing from send-order	correct behaviour	a pump will not take a new program while running
A device is connected but nothing is sent	it is not registered	Devices page, <i>Seen but not sent</i> , Register
Patient shows <i>conflict</i>	the allowed sources name different patients	read the detail; fix the ADT, clear the manual association, or change the device's sources
Patient shows <i>review</i>	the ADT patient in the bed changed while the device was running	check the bedside; Confirm new patient if it is right
"sign in to make changes"	an admin password is set	Sign in, top right
Save or Associate is greyed out	you are viewing from another computer with no password set	set a password, or use the gateway's own computer

Updating

Replace the files in your OpenBedside folder with the new version (or `git pull` if you cloned it), then:

```
source ~/.venvs/openbedside/bin/activate
pip install -e ~/Documents/OpenBedside
```

Windows

Install Python 3.11 or newer from python.org (tick "Add python.exe to PATH"). In PowerShell:

```
py -m venv $HOME\.venvs\openbedside
$HOME\.venvs\openbedside\Scripts\Activate.ps1
pip install -e "$HOME\Documents\OpenBedside[dev]"
openbedside demo
```

Removing it

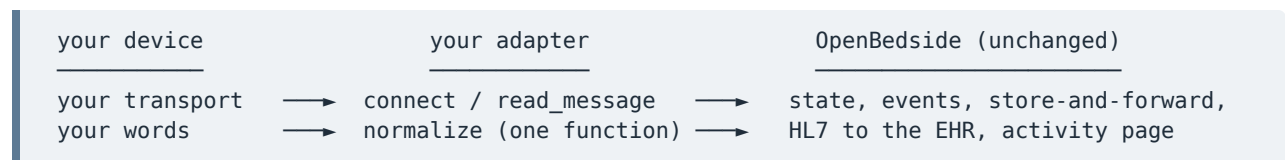
Delete the environment folder `~/ .venvs/openbedside` , the OpenBedside folder, and any `openbedside.db` files. Nothing else on the computer is changed by installing it.

Connecting your device to OpenBedside

A guide for the engineering team at a device manufacturer. By the end you will have your own device's data flowing through OpenBedside to a stand-in hospital interface engine, checked, tested, and ready to show to a hospital's interface team.

Plan on a few days for a first working adapter if your device already exposes its data somewhere. Most of that time goes into the mapping, not the code.

The idea



Every infusion device reports the same facts: what is running, at what rate, how much is left, what drug, what state. Every device says it differently: different transport, different state names, different unit strings, different ways of showing a piggyback. Your adapter is the translation, and it is the only code you write. Everything to the right of it is the same for every manufacturer, which is the point of the project.

What your device needs to provide

At minimum, from each pump module, repeatedly while connected:

- a stable device identifier and module identifier;
- the state (running, paused, stopped, complete, KVO, alarm);
- the current rate in mL/h when running.

Better, and what a hospital will ask for: drug name, concentration, programmed rate, VTBI, volume delivered, the piggyback when one runs, dose rate with its basis, and the device's own clock. Whatever your device cannot send, write down. Missing data is a finding the hospital needs to hear from you, not discover.

Step 1. Install OpenBedside

Follow [START-HERE-install-on-your-Mac.md](#). Linux is identical. Make sure `openbedside demo` runs and the page at <http://127.0.0.1:8080> shows the simulated pump.

Step 2. Look at a finished integration first

OpenBedside ships a complete adapter for a pump that does not exist, "ACME", with an invented protocol: JSON over TCP, its own state words, its own unit strings, two channels, piggybacks. Run it:

```
openbedside fake-pump           # tab 1: the pretend device
openbedside ehr-sim             # tab 2: the pretend hospital
openbedside run -c ~/Documents/OpenBedside/config/acme-example.toml # tab 3
```

Open <http://127.0.0.1:8080>. The **Device connections** card shows the adapter connected and counting messages, but nothing is sent yet: the pump is not registered. Go to **Devices**, find ACME-7731 under *Seen but not sent*, click **Register**, and save. From then on its data goes to the stand-in hospital. Stop the fake pump (Ctrl-C in tab 1) and watch the card change and a *communication-lost* event appear; start it again and watch the adapter reconnect on its own.

Then read `src/openbedside/examples/acme_json.py`. The adapter itself is about 110 lines (the rest of the file is the fake pump) and has three parts, which is the shape your adapter will have:

1. **Vocabulary tables:** `STATE_MAP` and `UNIT_MAP`, the device's words to ours.
2. **Transport:** `connect`, `read_message`, `close`.
3. **Translation:** `normalize`, one raw message in, one snapshot out.

Step 3. Fill in the mapping worksheet

`openbedside new-adapter` (next step) puts a copy in your folder as `MAPPING.md`, and it is also at [MAPPING-WORKSHEET.md](#). Do it before code. It asks about transport, identity, every state, every value and unit, how a piggyback appears, and what your device does not send. When it is done, the adapter is mostly copying it.

The rules it enforces:

- **Never guess.** Unknown stays empty. A plausible wrong number in a medical record is worse than a blank.
- **Every number has a unit.** OpenBedside refuses a number without one.
- **A dose rate travels with its basis:** concentration, and weight if weight based. Without them, OpenBedside withholds the dose rate rather than send something the hospital cannot check.
- **A two-channel pump is one device with two modules**, never two devices.
- **The secondary is a secondary.** While a piggyback runs, the primary waits, and the channel's actual rate is the secondary's.
- **Patient identity comes from the device or not at all.** If your device holds a patient identifier (the clinician scanned the wristband at the pump), put it in `reported_patient_id`. Never look one up, never set `Device.patient`. The gateway

decides the patient from the sources the hospital allows for that device and withholds it when they disagree.

- **The same device id and vendor string every time.** The hospital registers your device by vendor plus device id before anything from it is sent.

Step 4. Generate your starter folder

```
cd ~
source ~/.venvs/openbedside/bin/activate
openbedside new-adapter widgetco --dir ~/widgetco-adapter
```

You get:

File	What it is
<code>widgetco_adapter.py</code>	your adapter, with four marked sections to fill in
<code>widgetco.toml</code>	a config that runs it, with orders switched off
<code>test_widgetco_adapter.py</code>	tests that already pass on a sample message; grow them
<code>MAPPING.md</code>	the worksheet

The folder is yours. It does not need to live inside OpenBedside, and under the Apache license you may keep it private or publish it.

Step 5. Write the four sections

Your adapter subclasses `StreamAdapter`, which handles connecting, reconnecting with backoff, counting messages, skipping bad ones without crashing, and publishing. You write:

Method	Job
<code>connect()</code>	open the connection to the device, its test harness, or your own server
<code>read_message()</code>	return the next raw message, or <code>None</code> when the connection closes
<code>normalize(raw)</code>	return a <code>Device</code> snapshot, or <code>None</code> to skip a message that is not a status report
<code>close()</code>	tidy up (optional)

Transport patterns

The device (or your server) listens on TCP and streams. This is the template as written. Change the framing in `read_message` if yours is not one JSON object per line.

The device connects to the gateway. Write `run()` yourself with a server:

```

async def run(self):
    async def on_device(reader, writer):
        while line := await reader.readline():
            dev = self.normalize(json.loads(line))
            if dev:
                self.publish(dev)
    server = await asyncio.start_server(on_device, "0.0.0.0", int(self.config["listen_port"]))
    async with server:
        await server.serve_forever()

```

Serial (RS-232 or USB serial). Install `pyserial-asyncio` into your environment and open the port in `connect()`:

```

import serial_asyncio
async def connect(self):
    self.reader, self.writer = await serial_asyncio.open_serial_connection(
        url=self.config["port"], baudrate=int(self.config.get("baud", 9600))
    )

```

Your company already has a device server with an API. Poll it and publish each device. Use a thread for a blocking HTTP library so the gateway stays responsive:

```

async def read_message(self):
    await asyncio.sleep(float(self.config.get("poll_seconds", 5)))
    return await asyncio.to_thread(fetch_status_json, self.config["url"]) # your function

```

Binary protocols. `read_message` returns whatever `normalize` understands: bytes, a parsed structure, a dictionary. Keep the parsing in one place and test it with recorded messages (Step 8).

Things to get right

- **Publish on every report.** Do not re-send an old snapshot to look alive. When your device goes quiet, OpenBedside needs to notice and report communication lost.
- **Keep your vocabulary in the two tables.** Anything the tables do not cover should fail loudly (an unmapped unit raises an error and the message is skipped and counted), never be passed through as a guess.
- **Report the device clock** in `device_clock` if your device has one. OpenBedside warns when it drifts from the gateway, and event order depends on it.
- **Add units to `codes.UNITS` only with a source.** Each entry carries where its MDC code came from. If your device uses a unit that is not there yet, the checker tells you, and that is a contribution worth sending back.

Step 6. Check your mapping, before any EHR

```

openbedside check -c ~/widgetco-adapter/widgetco.toml --seconds 60 --show-hl7

```

This runs your adapter against your device (or its harness) for a minute, sends nothing anywhere, and reports:

- how many snapshots arrived, from how many devices, and how many messages were skipped;
- every problem, by level: **ERROR** (will be refused or mangled), **WARNING** (will work but could mislead), **INFO**;
- the longest silence from each device, against the offline threshold;
- with `--show-hl7`, the exact PCD-01 message a hospital would receive.

It ends with **PASS** or **FAIL**. Get to PASS with no errors before Step 7.

Finding	What it means	Usual fix
wrong-unit	a rate is not mL/h, or a volume is not mL	convert in <code>normalize</code>
unit-not-mappable	no MDC code for that unit yet	map to a supported unit, or add one to <code>codes.UNITS</code> with its source
status-unknown	a device state is missing from <code>STATE_MAP</code>	add it
no-actual-rate	running or KVO with no current rate	send the rate the channel is actually delivering
dose-without-basis	dose rate with no concentration	send concentration too, or leave dose rate empty
dose-without-weight	weight-based dose with no weight	send patient weight in kg
channels-per-module	more than one delivery channel in one module	model each channel as its own module
clock-skew	device clock far from the gateway's	fix the device time source
patient-set-by-adapter	the adapter set <code>Device.patient</code>	use <code>reported_patient_id</code> , and only for an id the device itself holds
pcd01-build-failed	the HL7 message could not be built	read the message; usually a unit or structure problem

The same checks are available in your tests: `openbedside.validate.check_device(dev)`.

Step 7. Run against the stand-in hospital and watch the activity page

```
openbedside ehr-sim # tab 1
openbedside run -c ~/widgetco-adapter/widgetco.toml # tab 2
```

On <http://127.0.0.1:8080>, first register your device on the **Devices** page (it appears under *Seen but not sent*). Then:

Card	What to confirm
Status	EHR link connected; queue shows messages <i>sent</i>
Device connections	your adapter connected; messages rising; skipped at 0; no last error
Devices	one row per module, the right status, rates and volumes that match the device's own screen
Events	each thing you do at the device appears once, in order, at the right time
Outbound to EHR	every message <i>sent</i> with AA

Tab 1 prints every message as the hospital receives it. Keep it next to the device.

Step 8. Test the hard cases

Work through this list at the device, with the activity page open. Write down what you see for each. These are the situations that break integrations in hospitals.

#	Do this	Expect
1	Start an infusion	<i>delivery-start</i> ; status infusing; rate matches the device
2	Change the rate	<i>rate-change</i> with old and new values
3	Pause, then resume	<i>delivery-stop</i> , then <i>delivery-start</i>
4	Start a piggyback	a secondary source appears; actual rate becomes the secondary's
5	Let the piggyback finish	<i>secondary-to-primary</i> ; actual rate returns to the primary's
6	Let VTBI run out	<i>delivery-complete</i> and <i>kvo-start</i> ; status KVO, still infusing
7	Cause an alarm	status alarm (alarm messaging to clinicians is out of scope)
8	Power the device off	after <i>offline_after_seconds</i> , <i>communication-lost</i> ; device marked offline
9	Power it back on	adapter reconnects; <i>communication-restored</i>

#	Do this	Expect
10	Stop <code>ehr-sim</code> for two minutes, then restart it	queue grows while it is down, then drains in order (retries back off, so allow a few minutes)
11	Stop and restart the gateway mid-infusion	nothing already queued is lost
12	Set the device clock 10 minutes wrong	the checker reports <code>clock-skew</code>
13	Run two channels with different drugs	each module keeps its own drug; nothing crosses over
14	<code>openbedside ehr-sim --reject-every 5</code>	every fifth message retried, none lost, order kept
15	Run the device before registering it	listed under <i>Seen but not sent</i> ; nothing reaches the EHR
16	If your device holds a patient id: scan one	the Patients page shows it with source <i>device</i> ; PID-3 carries it
17	Register the device to a bed with ADT on, then <code>openbedside send-adt --mrn TEST0001 --unit ... --bed ...</code> with a different id than the device reports	<i>association-conflict</i> ; PID-3 is <code>Unknown</code> until they agree

Then turn what you learned into automated tests. Record real messages from your device into a file and replay them through `normalize` in `test_widgetco_adapter.py`, one test per situation above. When your device firmware changes, run the tests first.

Step 9. Show it to a hospital

When your checklist is complete, a hospital interface team will want:

- [HL7-INTERFACE.md](#): exactly what they will receive, with samples;
- your completed `MAPPING.md`, including what your device does not send;
- the output of `openbedside check --show-hl7` against your device;
- your Step 8 results.

Connect to their **test** interface engine only, with synthetic patients. OpenBedside has no encryption on its HL7 links or its web page yet, and its database is not encrypted. Until that is done, keep it on a test network and keep the page on localhost.

And again: OpenBedside is not a medical device and is not for clinical use. Anyone who takes it into a product takes on that product's regulatory obligations.

Extending to ventilators and other devices

The pieces that are the same for any bedside device stay exactly as they are: the adapter pattern, `StreamAdapter`, the outbox, delivery to the EHR, the activity page, the checker's structure.

What changes for a new device class:

1. **The data model** (`model.py`): a ventilator has settings and measured values (mode, set and measured tidal volume, rate, PEEP, FiO2, pressures) rather than infusion sources.
2. **The codes** (`codes.py`): the device class, module and metric codes for that device, from the IHE Devices Technical Framework (DEV TF-3 Rev 10.0, section 7.2, "Device: Ventilator", which tracks the IEEE 11073-10303 ventilator work) and the Rosetta Terminology Mapping, each with its source.
3. **The HL7 builder**: the containment tree for that device.
4. **A simulator**, so everyone can test without hardware.
5. **Checker rules** for that device's values and units.

The outbound message does not change: IHE's PCD-01 is not pump-specific, and DEV TF-1 names ventilators among the devices it carries. Registration already accepts a ventilator; the gateway holds its data until a ventilator data model exists, rather than send it dressed as a pump. Ventilator support is planned. If your team builds it first, it is exactly the kind of contribution this project exists for.

Keeping in step

- Update OpenBedside: pull or download the new version, then `pip install -e ~/Documents/OpenBedside` again.
- Your adapter depends only on `openbedside.model`, `openbedside.adapters.stream` and `openbedside.validate`. Those are kept stable; changes will be listed in `CHANGELOG.md`.
- Found a problem in OpenBedside itself, or added a unit, a check or a transport pattern others could use? See [CONTRIBUTING.md](#).

Mapping worksheet

Fill this in before writing code. Every infusion device carries the same facts in different words. This page is where your words meet OpenBedside's. When it is complete, the adapter is mostly copying it into two tables and one function.

Device: _____ Firmware / protocol version: _____

1. Transport

Question	Your answer
How does data leave the device? (serial, TCP, UDP, vendor server API, file)	
Who starts the connection, the device or the gateway?	
Message framing (one JSON per line, length prefix, HL7, binary)	
How often does the device report while infusing? While idle?	
Does it send on change, on a timer, or only when asked?	
What happens when the connection drops? Does it buffer and resend?	
Authentication or encryption on the link	

2. Identity and containment

OpenBedside field	Meaning	Your field / how derived
<code>device_id</code>	stable, unique per physical device (EUI-64 or serial). The hospital registers the device by vendor plus this id	
<code>vendor</code> , <code>model</code>	shown on the status page and in HL7; <code>vendor</code> must be the same string every time	

OpenBedside field	Meaning	Your field / how derived
reported_patient_id	a patient identifier the device itself holds (a wristband scanned at the pump), or None. Never looked up or guessed by the adapter	
identifier the nurse scans	what is on the label the hospital scans to tie the device to a patient: serial, asset tag, EUI-64?	
module_id	stable per pump module or channel	
modules per device	a two-channel pump is ONE device with TWO modules	
device_clock	the device's own time, if it reports it	

3. States

Your state word	Meaning on your device	OpenBedside <code>PumpStatus</code>
	infusing at the programmed rate	INFUSING
	keep vein open after VTBI reached	KVO (still infusing)
	paused or on hold by a clinician	PAUSED
	stopped, no program	IDLE
	program finished	COMPLETE
	any alarm state	ALARM

Anything not listed maps to `UNKNOWN`, and the checker warns about it.

4. Values and units

OpenBedside field	Unit(s) allowed	Your field	Your unit string
actual rate (what the channel is doing now)	mL/h		

OpenBedside field	Unit(s) allowed	Your field	Your unit string
programmed rate, per source	mL/h		
VTBI	mL		
volume delivered	mL		
volume remaining	mL		
drug name	text		
concentration	mg/mL, [iU]/mL, ...		
dose rate	ug/kg/min, [iU]/h, ...		
patient weight	kg		

Rules: a number never travels without its unit; unknown is left empty; a dose rate is only useful with the concentration (and weight, if weight based) that produced it.

5. Primary and secondary

Question	Your answer
How does the device show a piggyback (secondary) is programmed?	
While the secondary runs, which rate does it report as "current"?	
Does the primary resume automatically when the secondary finishes?	

6. Things OpenBedside will infer, and whether your device tells you directly

Event	Device reports it? (message / field)
delivery start	
delivery stop	
delivery complete	

Event	Device reports it? (message / field)
rate change	
KVO start	
secondary to primary	

OpenBedside infers all of these from snapshots and labels them *derived*. If your device reports them directly, note it: a future release can pass device-reported events through as such.

7. What your device does NOT send

List anything a clinician would expect in the record that your device cannot supply (for example: drug name, concentration, weight, alarms). Missing data is a finding, not a detail. It goes in your integration notes to the hospital.

Adapter reference

The step-by-step guide is [VENDOR-INTEGRATION-GUIDE.md](#). This page is the short reference.

What an adapter may import

These are kept stable:

Module	What you use
<code>openbedside.adapters.stream</code>	<code>StreamAdapter</code> : subclass it, write <code>connect</code> , <code>read_message</code> , <code>normalize</code> , <code>close</code>
<code>openbedside.adapters.base</code>	<code>DeviceAdapter</code> , if you need full control of <code>run()</code>
<code>openbedside.model</code>	<code>Device</code> , <code>Module</code> , <code>Channel</code> , <code>InfusionSource</code> , <code>Quantity</code> , <code>PumpStatus</code> , <code>SourceRole</code> , <code>utcnow</code>
<code>openbedside.validate</code>	<code>check_device(dev)</code> for your tests; levels <code>ERROR</code> , <code>WARNING</code> , <code>INFO</code>

The snapshot

```
Device      device_id, vendor, model, observed_at, device_clock, patient_weight,
            reported_patient_id (only an id the device itself holds), kind
├─ Module   module_id (one per pump module or channel)
│   └─ Channel channel_id, status, actual_rate (exactly one per module)
│       └─ InfusionSource role PRIMARY or SECONDARY, drug_name, concentration,
                           rate, dose_rate, vtbi, vtbi_remaining, volume_delivered
```

`Quantity(value, ucum)` . Supported UCUM units are the keys of `codes.UNITS` : `mL` , `mL/h` , `mg` , `mg/mL` , `ug/kg/min` , `[iU]/mL` , `[iU]/h` , `kg` . Rates are mL/h and volumes are mL.

`PumpStatus` : `INFUSING` , `KVO` , `PAUSED` , `IDLE` , `COMPLETE` , `ALARM` , `UNKNOWN` .

What an adapter must not set

`Device.patient` , `ehr_id` , `eui64` and `location` are set by the gateway from the device registry and the association step. `openbedside check` reports an `ERROR` if an adapter sets `patient` . A device the hospital has not registered is held, not sent.

Registering an adapter

```
[gateway]
adapter_paths = ["."]          # folders to search, relative to this config file

[[adapters]]
type = "mymodule:MyAdapter"    # module:ClassName
# any other keys are yours, available as self.config["..."]
```

Programming

Real-device adapters do not receive programming requests in this release. Leave `supports_programming` `False` and set `[orders] enabled = false`.

HL7 interface

What the hospital's interface engine receives from OpenBedside, and what it may send.
Written for the interface analyst. HL7 v2.6, MLLP framing, UTF-8.

Connections

Direction	Transaction	Message	Default	Acknowledgement
Gateway → EHR	IHE PCD-01, Communicate PCD Data	ORU^R01^ORU_R01	client to 127.0.0.1:6661	application ACK required (MSH-15 NE , MSH-16 AL)
EHR → Gateway	IHE PCD-03, Communicate Infusion Order	RGV^015^RGV_015	listens on 127.0.0.1:6662	RRG^016 on the same connection. Simulator only.
EHR → Gateway	HL7 v2 ADT (patient location)	ADT^A01 , A02 , A03 , A04 , A08 , A11 , A12 , A13 , A40	listens on 127.0.0.1:6663	ACK , AA for every event, used or not
Gateway → EHR	IHE PCD-10, Infusion Event	ORU^R42^ORU_R01	planned	

Ports and hosts are set in the config file. Nothing listens on a network interface other than localhost until someone changes that on purpose.

When PCD-01 is sent

Only for devices registered on the Devices page (unless [registry] unknown_devices = "send"), and then:

- whenever the engine derives an event on any channel of a device;
- whenever the device's patient, location or registration changes;
- otherwise every periodic_seconds (default 60) per device.

Each message is a full snapshot of the device, not a delta.

Containment in OBX-4

MDS.VMD.CHANNEL.METRIC . For module *m*:

OBX-4	Meaning
1.0.0.0	the device (MDS). OBX-18 identifies it, see below
1.m.0.0	module <i>m</i> (VMD). OBX-18 carries the module id

OBX-4	Meaning
1.m.1.0	delivery channel: what the module is doing now
1.m.2.0	primary source: what is hanging
1.m.3.0	secondary source, only when a piggyback is programmed

Metrics inside a delivery channel: .1 status, .2 actual flow rate. Metrics inside a source: .1 drug name, .2 concentration, .3 programmed rate, .4 dose rate, .5 volume to be infused, .6 volume remaining, .7 volume delivered, .8 patient weight (only when the dose rate is weight-based).

Device identity: OBX-18

identifier^vendor, plus ^EUI-64 value^EUI-64 when the device's registration has an EUI-64. The identifier is whatever the registration says the hospital scans and files by: the serial number, the hospital asset tag, or the EUI-64. Module ids follow it: BIO-004512-L for the left module of a pump registered by asset tag BIO-004512. This follows the form in published vendor PCD-01 specifications; check it against DEV TF-2 Appendix B before claiming conformance.

Patient: PID and PV1

PCD-01 requires a PID segment (DEV TF-2 Rev 10.0, 3.1.4.1.1, PID R [1..1]), so there is always one. The Unknown form below is the one used in published vendor PCD-01 specifications; confirm it is what your interface engine expects.

Situation	PID-3	PV1
patient associated	id^^^authority^MR	PV1-2 class, PV1-3 unit^room^bed^facility, PV1-19 visit, from the ADT census when the patient is in it
no patient, device has a registered location	Unknown^^^^U	PV1-2 U, PV1-3 the device's registered location
no patient, no location	Unknown^^^^U	none

PID-5 is always ^^^^^U (name type unspecified). The gateway does not store or send patient names.

How the gateway decides the patient, per device, from the sources its registration allows:

- **device:** the device reported an identifier (a wristband scanned at the pump);
- **manual:** a person associated them on the Patients page;
- **adt-location:** the ADT census has exactly one patient in the device's bed.

All allowed sources that have an answer must agree, or PID-3 is `Unknown` and the conflict is shown on the Patients page. If ADT location is the only source and would change the patient while the device is running, PID-3 is `Unknown` until a person confirms. The *patient-associated*, *patient-cleared*, *association-conflict* and *association-review* events on the activity page record every decision.

Rules the receiver can rely on

- **The patient is never guessed.** Unknown is sent as Unknown.
- **Dose rate travels with its basis.** No concentration (or no weight for a weight-based dose) means no dose rate is sent.
- **Units are MDC codes with UCUM alternates** in OBX-6.
- **Timestamps carry a UTC offset.**
- **Codes are labelled.** Every MDC code used is listed in `codes.py` with its source and status. None is yet marked verified against the current RTM tables.

Sample PCD-01 from the simulator

Registered to ICU 101 A with association by ADT location, after a synthetic admit of TEST0001. First eight segments; the rest are as before.

```
MSH|^~\&|OpenBedside|GATEWAY|EHR|HOSP|20260926142209+0000||ORU^R01^ORU_R01|PD2026092614220900001|P|2.6|||NE|AL|||IHE_PCD_ORU_R01^IHE_PCD^1.3.6.1.4.1.19376.1
PID|||TEST0001^^^HOSP^MR|^^^^^U
PVI||Z|ICU^101^A||||||V0001
OBX|1||PD2026092614220900001^OpenBedside|69985^MDC_DEV_PUMP_INFUS_MDS^MDC|||20260926142209+0000
OBX|1||69985^MDC_DEV_PUMP_INFUS_MDS^MDC|1.0.0.0|||||X|||20260926142209+0000|||SIM-0001^OpenBedside-Sim
OBX|2||69986^MDC_DEV_PUMP_INFUS_VMD^MDC|1.1.0.0|||||X|||20260926142209+0000|||SIM-0001-A^OpenBedside-Sim
OBX|3||126978^MDC_DEV_PUMP_INFUS_CHAN_DELIVERY^MDC|1.1.1.0|||||X|||20260926142209+0000|||
OBX|4||CWE|184508^MDC_PUMP_STAT^MDC|1.1.1.1|^pump-status-infusing^MDC|||||R|||20260926142209+0000|||
...
```

Sample PCD-03 accepted by the simulator

```
MSH|^~\&|TestEHR|TESTFAC|OpenBedside|GATEWAY|20260926142209+0000||RGV^015^RGV_015|T02026092614220900001|P|2.6|||AL|AL|||IHE_PCD_RGV_015^IHE_PCD^1.3.6.1.4.1.1
PID|||TEST0001^^^HOSP^MR
ORC|RE|ORD0001^TestEHR
RXG|1||^heparin|100||263762^MDC_DIM_MILLI_L^MDC|10|265266^MDC_DIM_MILLI_L_PER_HR^MDC
RXR|IV
OBX|1||69986^MDC_DEV_PUMP_INFUS_VMD^MDC|1.1.0.0|||||F||||SIM-0001-A
```

The target module is identified by OBX-18 on the OBX whose OBX-3 is `69986` (VMD). The rate is read from RXG-15 with units in RXG-16 (mL/h only in this release), the volume to be infused from RXG-5.

Reply codes in `RRG^016` MSA-1: `AA` accepted by the simulator; `AE` could not be done (unknown module, module infusing, parameter missing); `AR` refused (not a PIV message, wrong version, or the target is not the simulator).

ADT the gateway accepts

Identifiers and location only are kept: PID-3 (with PID-3.4 as the assigning authority), PV1-2, PV1-3 and PV1-19. Names and demographics are ignored and never stored.

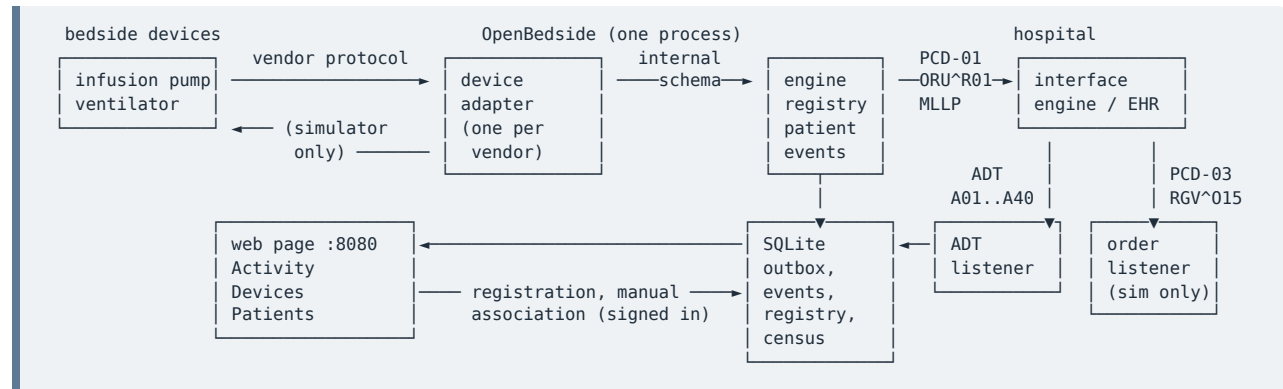
Event	Effect on the census
A01 admit, A04 register, A02 transfer, A13 cancel discharge	patient is at PV1-3
A08 update	PV1-3 if present, otherwise the location already held
A03 discharge, A11 cancel admit	patient leaves the census
A12 cancel transfer	patient returns to PV1-6 (prior location)
A40 merge	MRG-1 becomes PID-3; manual associations follow

Every ADT gets an `ACK` with AA, including events the gateway does not use, so the interface engine does not hold them. A message without PID-3 gets AE.

Sample, as sent by `openbedside send-adt --mrn TEST0001 --unit ICU --room 101 --bed A --visit V0001`:

```
MSH|^~\&|TestADT|TESTFAC|OpenBedside|GATEWAY|20260926174324+0000||ADT^A01^ADT_A01|AD2026092617432400004|P|2.6
EVN|A01|20260926174324+0000
PID|1||TEST0001^^^HOSP^MR||TEST^PATIENT
PV1|1|I|ICU^101^A|||||||||||||V0001
```

Architecture



The one rule

Vendor protocols stop at the adapter. Everything to the right of the adapter reads only `model.py`. That is what makes the rest identical for every manufacturer, and it is what lets any device be connected by writing one class instead of a gateway.

Pieces

Module	Job	Knows about
<code>model.py</code>	internal schema: Device (MDS) → Module (VMD) → Channel → Source	nothing else
<code>adapters/</code>	talk to devices, publish Device snapshots; <code>StreamAdapter</code> does the connect, read, reconnect loop	one vendor's protocol
<code>validate.py</code>	check snapshots before a hospital sees them (<code>openbedside check</code>)	the schema, the HL7 builder
<code>scaffold.py</code>	write a starter adapter folder (<code>openbedside new-adapter</code>)	nothing
<code>examples/</code>	the fictional ACME pump and its adapter, a complete worked integration	its own invented protocol
<code>core/engine.py</code>	check registration, keep state, derive events, decide what to send	the schema, the outbox, the registry
<code>core/association.py</code>	decide each device's patient, and say how	the registry and census
<code>store/registry.py</code>	registered devices, ADT census, manual associations, admin password	SQLite

Module	Job	Knows about
<code>ehr/adt.py</code>	accept ADT, keep the census (identifiers and location only)	HL7
<code>store/outbox.py</code>	durable queue and event log	SQLite
<code>hl7/</code>	build and parse HL7 v2, MLLP framing, ACKs	HL7, MDC codes
<code>ehr/sender.py</code>	deliver the queue, in order, on application ACK	MLLP
<code>orders/piv.py</code>	accept PCD-03, validate, route (simulator only)	HL7, adapters
<code>ui/server.py</code>	Activity, Devices and Patients pages; sign-in; JSON API	the state snapshot, the registry

Server and interface in one

Earlier commercial gateways split the device server from the interface engine and ran a full SQL database server, for reasons of history as much as design. Here they are one process, and the database is a single SQLite file next to it. A hospital, or a developer on a laptop, installs one thing.

Delivery guarantees

- Every outbound message is written to disk before it is sent.
- A message is delivered only when the receiver returns an application ACK (AA or CA) that names its control id.
- Order is kept per destination. If the oldest message cannot be delivered, nothing behind it goes out.
- An explicit reject (AR) is set aside for a person to review and the queue moves on.
- Everything else backs off and retries, up to five minutes between tries.

Events: what is observed and what is inferred

IHE's infusion event profile (IPEC) requires three events: delivery start, stop and complete. It has **no** rate change, **no** secondary-to-primary switchover and **no** transition to KVO, which are the states a clinician needs to read a pump record. The engine infers all of these from successive snapshots and labels every one `derived`, so nothing downstream mistakes an inference for a device report.

Cloud later, without redesign

The outbox is addressed by destination. A second sender can drain the same queue to an HTTPS endpoint, a message broker or a cloud service, alongside or instead of the EHR. Nothing in the adapters or the engine changes. Whether to do that is a market question this project will

answer when hospitals do.

Traps the design handles on purpose

- **Rate is not dose.** A dose rate is sent only with the concentration, and weight when weight-based, that produced it. Missing basis means the dose rate is withheld.
- **KVO is still infusing**, at a rate unrelated to the order. It is its own status.
- **A secondary pauses the primary** on a shared line. Only the active source's rate is the channel's actual rate.
- **Multi-module pumps are one device.** Two modules are two VMDs under one MDS.
- **Patient identity** is never guessed. It comes from sources the registration allows (the device, a person, the ADT census for the device's bed), and a disagreement means `Unknown`. See `core/association.py`.
- **Registration before transmission.** An unregistered device is held and shown on the Devices page, not sent. Identity is vendor plus device id, never an IP address.
- **Clocks drift.** The model keeps both the gateway's observation time and the device's own clock so skew can be seen.

Decisions

Recorded so nobody has to reconstruct why. Newest first.

2026-09-26: Patient association is in scope (reverses "out of scope")

Without a patient, device data is incomplete even for status: the hospital cannot file it. So the gateway decides the patient, from authoritative sources only, and records which: the device's own report (a wristband scanned at the pump), a manual association on the Patients page, or the ADT census for the device's registered bed. All allowed sources must agree or nothing is sent. Two patients in one bed is a conflict. A patient change from ADT alone while the device is running waits for a person, because that is the wrong-patient case: the pump went with the patient and someone new arrived in the old bed.

Privacy: using patient identifiers to file device data is treatment, which HIPAA permits without authorization (45 CFR 164.506). That settles whether the gateway may use them. It does not remove the Security Rule's safeguards for whoever runs the gateway, and a business associate agreement is needed when someone other than the hospital operates it. This release stores identifiers and location only, no names, and is for synthetic data until encryption and access logging are done.

2026-09-26: PID is always sent

DEV TF-2 Rev 10.0 (3.1.4.1.1) lists PID as required in PCD-01. 0.1 omitted it for an unassociated device; that was wrong. An unassociated device now sends

`PID|||Unknown^^^^U||^U`, the form used in published vendor specifications.

2026-09-26: Devices are registered before anything is sent

A device is known by vendor plus device id (two vendors can both ship serial 12345). Its registration says what type it is, which identifier the hospital scans (serial, asset tag or EUI-64), where it lives, and which association sources are allowed. Unregistered devices are held and shown, not sent, by default. IP addresses are never identity: DHCP moves them and one device server speaks for many devices. Registration is on the web page, so the page gained sign-in: an admin password, or with none set, changes only from the same computer.

2026-09-26: Why this exists, stated plainly

The author is building AI MedAgent, a medication reasoning engine that needs EHR data and bedside device data. Certified EHR technology must offer a standardized FHIR API (ONC, 45 CFR 170.315(g)(10)); bedside devices have no equivalent, and IV infusion pumps, among the

most important feeds for medication reasoning, report only through closed, manufacturer-specific gateways. OpenBedside opens that path for everyone. IV infusion comes first for that reason. The project serves any consumer of the data and does not depend on AI MedAgent in any way.

2026-09-26: Scope of 0.1: data out is real, orders are simulator only

FDA's MDDS guidance (final, 28 September 2022) treats software functions "solely intended to transfer, store, convert formats, or display medical device data and results" as outside device regulation, and keeps oversight of software that controls or alters a connected device and of alarms for active patient monitoring.

So: status reporting, store-and-forward, HL7 conversion and the status page are real in 0.1. Auto-programming (PCD-03) is built and tested end to end, but routes only to the simulator. Alarm communication (ACM) is not in scope for the open release.

A manufacturer that takes this code into its own product can build the regulated pieces under its own quality system. That is where they belong.

2026-09-26: License: Apache 2.0

The goal is an open device data path that anyone can use, so anyone must be able to use, modify and build it into their own products, without fees or permission. Apache 2.0 does that and adds an explicit patent grant, which covers only what the contributed code itself practices. This project is plumbing and contains no clinical reasoning; the author's AI reasoning work is kept out of this repository entirely.

2026-09-26: Clean room

This project is designed from public sources: IHE Devices Technical Framework, ISO/IEEE 11073 nomenclature as published, FDA guidance, the author's own experience, and the general form of publicly distributed customer-facing vendor specifications. No text, tables or code are copied from any vendor document, and no material received under any confidentiality agreement is used.

2026-09-26: One process, one file database

Server and interface engine are combined, and the store is SQLite. Earlier commercial gateways separated them and ran a database server, largely for historical reasons. One install is the point.

2026-09-26: Standard library only

No runtime dependencies. A hospital security review, or a developer on a laptop, has one thing to evaluate and nothing to update but Python itself.

2026-09-26: Derived events are labelled

IHE IPEC has no rate change, no secondary-to-primary switchover and no KVO transition. The engine infers them and marks each `derived`. A downstream system can always tell an inference from a device report.

2026-09-26: MDC codes carry their provenance

No code is typed from memory. `codes.py` records where each came from and a status. Units without a code refuse to be sent. Before production, every entry is to be checked against the current Rosetta Terminology Mapping and marked VERIFIED.

Open questions

- PCD-10 event codes: verify against DEV TF-2 Rev 10.0 Appendix M before building.
- Separate-connection application ACK for PCD-03, per IHE PIV.
- Location of mobile devices: association by ADT location only suits devices that stay put. RTLS or a bedside scan would tell the gateway where a device really is.
- PIV orders name the target module by the EHR's identifier; with asset-tag registration the simulator's order routing does not yet translate it back.
- Ventilator schema: DEV TF-3 Rev 10.0 section 7.2, "Device: Ventilator" (tracks the IEEE 11073-10303 work), and the ventilator terms in 11073-10101 as amended.
- Security: TLS for MLLP and the web page, database encryption, audit log of every access. Required before anyone points this at a real network or real patients.
- Name: "OpenBedside" confirmed by the author on 26 September 2026. A formal trademark search has not been done.

Contributing

OpenBedside is open source under the Apache License 2.0. Use it, change it, build it into your own products, extend it to other devices. Improvements that help everyone are welcome back.

What is most useful

- **Adapters, or pieces of them:** transport patterns, framing helpers, test harnesses. You do not have to publish your device's adapter; under the license you may keep it private. Generic pieces that would help the next manufacturer are gold.
- **Units and codes** in `codes.py`, each with the source it came from.
- **Checks** in `validate.py` for mistakes you made once and would like the next team not to make.
- **Ventilator support** and other device classes (see the vendor guide).
- **Verification** of the MDC codes against the current Rosetta Terminology Mapping.
- **Documentation** that would have saved you a day.

Ground rules

1. **No proprietary material.** Nothing copied from any manufacturer's confidential documents, code or protocol specifications, and nothing received under a non-disclosure agreement. Public standards and your own work only.
2. **No code from memory.** Every MDC code carries its source. Unverified stays labelled unverified.
3. **Never guess clinical data.** Unknown stays empty. Units always travel with numbers. A dose rate always travels with its basis.
4. **Standard library only** in the core. Adapters may use whatever their device needs.
5. **Tests with every change.** `pytest` must pass.
6. **No em dashes.** House style.

How

Until a public repository is announced, send changes or questions to the maintainer through danielpettus.com. By contributing you agree that your contribution is licensed under the Apache License 2.0, as the license itself provides.

Regulatory reminder

OpenBedside is not a medical device. Contributions that would make it control a real device (auto-programming beyond the simulator) or deliver alarms for active patient monitoring belong in a manufacturer's own regulated product, not here.

Disclaimer

OpenBedside is engineering software for testing, learning and development.

- **It is not a medical device** and has not been cleared, approved or validated by the FDA or any other regulator.
- **It is not for clinical use.** Do not connect it to devices in use on patients.
- **It is not clinical decision support.** It does not check doses, rates, drug libraries or orders for clinical appropriateness, and nothing it displays is dosing guidance.
- **Auto-programming works against the built-in simulator only.** Software that controls or alters a connected medical device is a regulated device function (FDA, *Medical Device Data Systems, Medical Image Storage Devices, and Medical Image Communications Devices*, final guidance, 28 September 2022). This release refuses to route programming requests to any adapter other than the simulator.
- **Patient association is decided only from sources the hospital allows** for each device: an identifier the device itself reports, a person's association on the Patients page, or the ADT census for the device's registered bed. When they disagree, no patient is sent. Association by location trusts that the device is where it is registered; the gateway cannot see where a device really is. Getting the right patient remains the hospital's workflow and the hospital's responsibility.
- **Use synthetic patient data only.** Once patient identifiers flow through it, the gateway stores and transmits protected health information, and the HIPAA Security Rule's safeguards apply to whoever operates it. This release has no encryption on its HL7 links or web page and no encryption of its database. Real patient data needs all three, plus access logging, first.
- The MDC codes in `codes.py` are labelled with their source and status. None is marked verified against the current Rosetta Terminology Mapping yet. Verify before relying on any of them.

Anyone who takes this code into a regulated product takes on the regulatory obligations for that product.